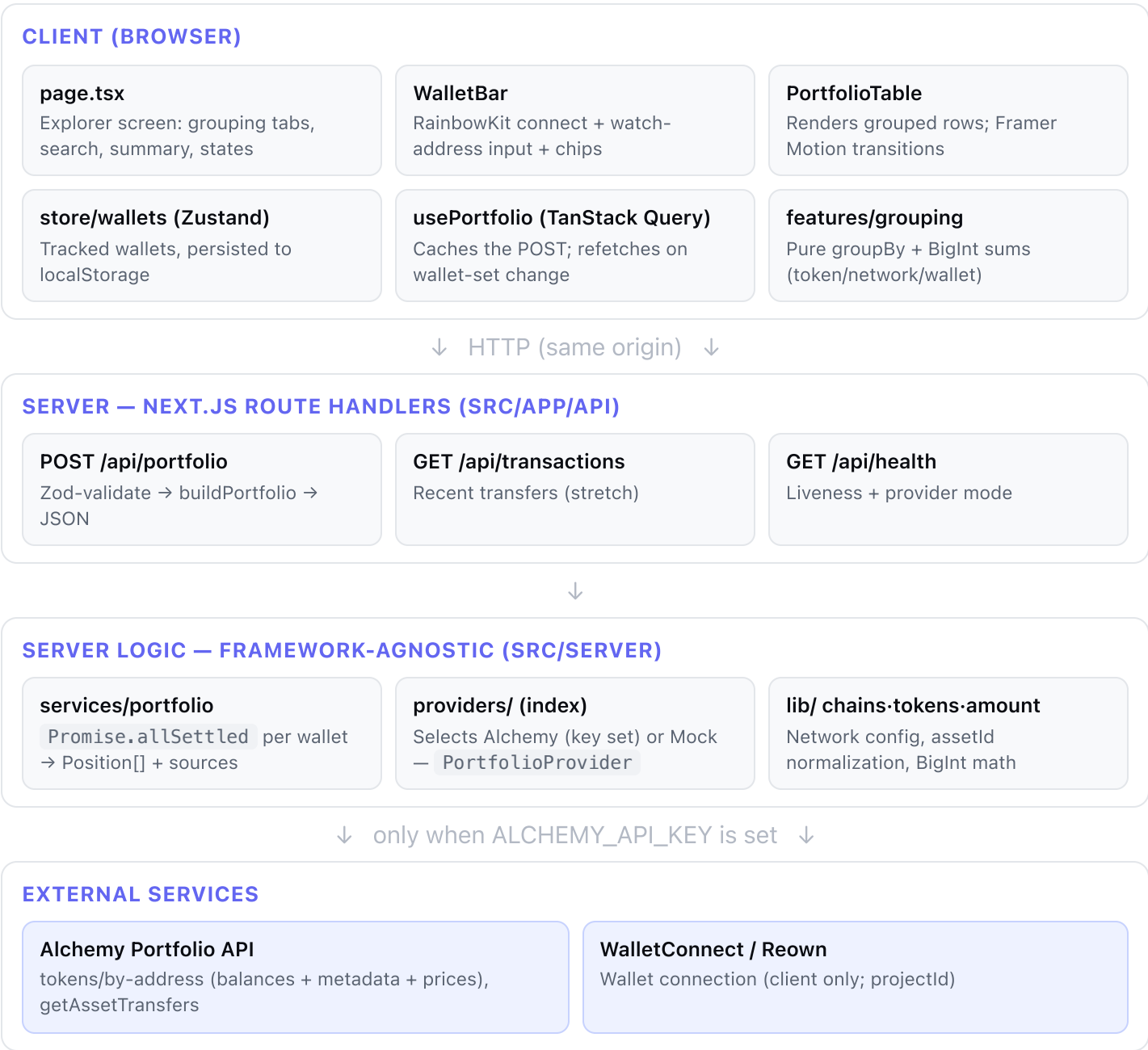


Multi-Chain Portfolio Explorer — System Design

One Next.js app: UI + API Route Handlers, no database. Deployed to Vercel.

Component layers



Key design decisions

- Flat Position[] + client-side grouping.** The server returns one row per (wallet × token); grouping is a pure transform. New groupings need no API change and switching never refetches.
- Provider abstraction with mock fallback.** `getProvider()` returns Alchemy or Mock by env. The app runs and is demoable with zero credentials, and swaps to a different API by adding one class.

BigInt for balances. Raw balances are integer base units (strings on the wire), summed as BigInt; conversion to a JS number happens only at display, avoiding float precision loss / overflow.

Layered token identity. Canonical `(chainId, address)` ; cross-chain group key via curated registry → native id → `symbol@decimals` . Symbol alone is never the key.

Partial failure as data. `allSettled` + a `sources[]` status array means a failing chain degrades gracefully instead of erroring the whole view.

Data shape

- **Position:** walletAddress, walletLabel, chainId, networkName, tokenAddress, **assetId**, symbol, decimals, rawBalance (string), priceUsd, valueUsd
- **SourceStatus:** walletAddress, status (ok|failed), count?, error?
- **PortfolioResponse:** positions[], sources[], mode (live|mock)

Deployment

- Single Vercel project, Root Directory `apps/web` — Next.js auto-detected.
- Env: `ALCHEMY_API_KEY` (server), `NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID` (client).
- No database, no separate API host. Route Handlers run as serverless functions.